

SYSTEM DESIGN

Complete Study Notes

Week 1 & Week 2 — Foundations + Databases

3rd Year CSE | MAANG Placement Prep

Table of Contents

01	How the Internet Works & APIs	Pg 3
02	TCP vs UDP	Pg 4
03	Scaling — Vertical vs Horizontal	Pg 5
04	Load Balancers	Pg 6
05	Caching — Redis, CDN, Eviction, Invalidation	Pg 7
06	Databases — Sharding	Pg 9
07	Databases — Replication	Pg 10
08	ACID vs BASE	Pg 11
09	SQL vs NoSQL at Scale	Pg 12
10	Interview Cheat Sheet	Pg 13

How the Internet Works & APIs

The Request-Response Cycle

When you type `instagram.com` in your browser, a chain of events fires in milliseconds:

```
Browser → DNS Server → gets IP address
→ HTTP Request → Instagram Server
→ processes request
← HTML / JSON response ←
Browser renders the page
```

Key Terms

Term	What it means	Example
DNS	Phonebook of the internet. Converts domain → IP	<code>nitsrinagar.com</code> → <code>142.250.x.x</code>
IP Address	Actual address of the server on the network	<code>142.250.80.46</code>
HTTP	Protocol for data transfer (plain)	Most web requests
HTTPS	HTTP + TLS encryption	Banking, login pages
Latency	Time for request to travel to server and back	50 ms Delhi→Mumbai
Bandwidth	How much data flows per second	100 Mbps connection

REST APIs

An API is a contract between two systems. REST is the most common style:

- Uses HTTP methods — GET, POST, PUT, DELETE, PATCH
- Stateless — server does NOT remember previous requests
- Resources are URLs — `/users/5`, `/posts/123`

HTTP Methods

Method	Purpose	Example
GET	Retrieve data	GET <code>/users/5</code> → fetch user 5
POST	Create new resource	POST <code>/users</code> → create user
PUT	Replace entire resource	PUT <code>/users/5</code> → update user 5
PATCH	Partial update	PATCH <code>/users/5</code> → change one field
DELETE	Remove resource	DELETE <code>/users/5</code> → delete user 5

■ GET is idempotent — calling it 100 times gives the same result, no side effects. POST is NOT idempotent — 100 calls create 100 records.

HTTP Status Codes (Memorise These)

Code	Meaning	Who is at fault?
------	---------	------------------

200	OK — success	—
201	Created	—
400	Bad Request	Client's fault
401	Unauthorized — not logged in	Client's fault
403	Forbidden — logged in but no permission	Client's fault
404	Not Found	Client's fault
500	Internal Server Error	Server's fault

TCP vs UDP

	TCP	UDP
Guarantee	Guaranteed delivery, ordered packets	No guarantee, no ordering
Speed	Slower (handshake + ACK overhead)	Faster (fire and forget)
Use case	Web, email, banking, file transfer	Live video, gaming, GPS tracking
Connection	Connection-oriented (3-way handshake)	Connectionless
Example	Loading a webpage	Google Maps live location

■ Interview Answer: For real-time location tracking (Uber/Swiggy), use UDP — speed matters more than guaranteed delivery. If one packet drops, we fall back to last known location. TCP's ACK overhead would cause noticeable lag. Same reason multiplayer games use UDP.

Scaling — Vertical vs Horizontal

The Problem

Your food delivery app starts with 1 000 users. Two years later — 50 million. One server can no longer cope. You need to scale.

Vertical Scaling (Scale Up)

Give the existing server more power — more RAM, more CPU cores, faster disk.

Before: [Server — 8 GB RAM, 4 cores]
 After: [Server — 64 GB RAM, 32 cores]

- Fast to implement — just buy bigger hardware
- Has a hard ceiling — you can't grow forever
- Single point of failure — server dies = everything dies
- Good for databases that are hard to distribute (early stage)

Horizontal Scaling (Scale Out)

Instead of one big server, add more servers and distribute load between them.

Before: [Server 1]
 After: [Server 1] [Server 2] [Server 3]
 ↑ ↑ ↑
 handled by Load Balancer

- Virtually unlimited — keep adding servers
- Fault tolerant — if one server dies, others continue
- Requires stateless servers + load balancer
- More complex to implement and manage

	Vertical	Horizontal
Cost	Expensive specialised hardware	Cheaper commodity machines
Limit	Has a ceiling	Virtually unlimited
Failure	Single point of failure	Fault tolerant
Complexity	Simple	Complex (needs LB, stateless design)
Real examples	Old school banks	Google, Netflix, Amazon

■■ Horizontal scaling only works if servers are STATELESS. Store sessions externally in Redis — not on the server — so any server can handle any request.

■ Interview Answer: Traffic 10x overnight? First vertically scale for immediate relief (faster to do). Simultaneously move to horizontal scaling — add load balancer, make servers stateless by moving sessions to Redis, spin up more instances. Long term, horizontal is correct because vertical has a ceiling and is a single point of failure.

Load Balancers

What is a Load Balancer?

Sits in front of all your servers and distributes incoming traffic across them.

```
→ [ Server 1 ]  
[ User ] → [ LB ] → [ Server 2 ]  
→ [ Server 3 ]
```

Load Balancing Algorithms

Algorithm	How it works	Best for
Round Robin	Rotate: S1 → S2 → S3 → S1...	Equal request sizes
Least Connections	Send to whichever server is least busy now	Variable request sizes (file uploads)
IP Hashing	Same user always goes to same server	When some server state is unavoidable
Weighted RR	Servers with more power get more traffic	Mixed hardware setups

Health Checking

Load balancer pings each server periodically. If a server doesn't respond → removed from rotation automatically. Users never notice.

```
[ Server 1 ] ■ – healthy  
[ Server 2 ] ■ – LB stops sending here  
[ Server 3 ] ■ – healthy
```

■■ TRAP: What if the Load Balancer itself crashes? Run multiple LBs in Active-Passive failover. If the active LB dies, the passive one takes over instantly.

Caching — Redis, CDN, Eviction, Invalidation

What is Cache in System Design?

Cache is a temporary storage layer that saves results of expensive operations so you don't repeat them.

WITHOUT cache:

User → Server → DB query (200 ms) → return result

WITH cache:

User 1 → Server → DB (200 ms) → save in Redis

User 2 → Server → Redis (2 ms) → return instantly ■

■ **80-20 Rule: 80% of traffic accesses only 20% of data. Cache that 20% and your DB load drops dramatically.**

Why NOT Cache Everything?

- RAM is expensive and limited — can't store terabytes
- Data goes stale — cached score won't update when match changes
- Not all data is equally popular — caching rarely accessed data wastes space

Redis

Key-value store that lives in RAM. Extremely fast (~0.1 ms). Used by Twitter, GitHub, Instagram.

Key Value

"user:5" → { name: "Arjun", age: 21 }

"otp:9988" → "482910" (TTL: 300s)

"score" → "IND: 287/4"

- Session storage — keeps login state external so any server can access it
- Rate limiting — count API calls per user per minute
- Leaderboards — sorted sets for real-time rankings
- Pub/Sub — message broadcasting

CDN (Content Delivery Network)

Cache for STATIC files (images, videos, JS, CSS). Copies placed at servers geographically close to users.

Without CDN:

User in Delhi → Netflix server in USA → high latency ■

With CDN:

User in Delhi → CDN node in Mumbai → fast load ■

Providers: Cloudflare, Akamai, AWS CloudFront. Netflix serves 15% of global internet traffic through CDN.

Cache Eviction Policies

When cache is full, what gets deleted to make space?

Policy	Full Form	Logic	Use when
LRU	Least Recently Used	Evict what wasn't accessed longest	General purpose — most common

LFU	Least Frequently Used	Evict what is accessed least often	When popularity is stable
FIFO	First In First Out	Evict oldest entry	Simple queues
TTL	Time To Live	Auto-delete after fixed time	OTPs, sessions, scores, news

■ For time-sensitive data (trending posts, live scores, OTPs) — always use TTL. For general data — LRU is the safe default.

Cache Invalidation

When real data changes in DB, how do you keep cache consistent? This is famously one of the hardest problems in CS.

Strategy 1 — Write Through

User updates name
 → Write to DB ■
 → Write to cache simultaneously ■
 → Always consistent, slightly slower writes

Strategy 2 — Cache-Aside (Most Common)

User updates name
 → Write new name to DB ■
 → DELETE the cache entry (don't update, DELETE)
 → Next request: cache miss → fetch fresh from DB → store in cache

■■ Never update cache before DB. If server crashes between the two operations, cache has new data but DB has old data. DB is always the source of truth.

Full Request Flow (Everything Together)

User types URL
 ↓
 DNS → gets IP address
 ↓
 Load Balancer → picks a server
 ↓
 Server → checks Redis cache first
 ↓
 Cache HIT → return immediately ■
 Cache MISS → query Postgres / MongoDB
 → store result in Redis
 → return to user
 ↓
 Static files (images/video) → served from CDN ■

Databases — Sharding

The Problem

500 million users. A single database server can't hold all that data or handle all that traffic. You need to split the data itself across multiple servers.

What is Sharding?

Sharding = splitting your data horizontally across multiple DB servers. Each server holds a subset and is called a shard.

Without sharding:

[All 500M users → Server 1] ← overloaded ■

With sharding:

[Users 1–125M → Shard 1]

[Users 125–250M → Shard 2]

[Users 250–375M → Shard 3]

[Users 375–500M → Shard 4]

Shard Key — The Most Important Decision

Range-Based Sharding

Split by ranges of a field (e.g. user ID 1–125M on Shard 1).

■■ Problem: Hotspot. New users always get high IDs → Shard 4 gets all new signups. Other shards sit idle.

Hash-Based Sharding (Preferred)

Shard Number = $\text{hash}(\text{user_id}) \% \text{number_of_shards}$. Traffic is distributed evenly and randomly.

User 1001 → $\text{hash}(1001) \% 4 = 1$ → Shard 1

User 1002 → $\text{hash}(1002) \% 4 = 2$ → Shard 2

User 1003 → $\text{hash}(1003) \% 4 = 3$ → Shard 3

■ Instagram shards by user_id hash. Discord shards by server_id hash.

Trade-offs of Sharding

Problem	Explanation
Cross-shard queries	Querying data across multiple shards requires hitting all of them and merging — slow and complex
Resharding pain	Going from 4 → 8 shards means almost all data moves. Extremely difficult without downtime
No cross-shard ACID	Atomic transactions across two shards (e.g. bank transfer) are nearly impossible to guarantee
Hotspot risk	Bad shard key choice means one shard gets all traffic while others idle

■■ Sharding is a last resort. Exhaust indexing → caching → read replicas → vertical scaling BEFORE sharding.

Databases — Replication

What is Replication?

Replication = keeping exact copies of your database on multiple servers. Serves two very different purposes.

Purpose 1 — Fault Tolerance

Primary DB crashes ■
→ Replica automatically promoted to Primary
→ Zero downtime for users ■

Purpose 2 — Read Scaling (Most Important for Interviews)

Most apps have ~80% reads and ~20% writes. Send reads to replicas. Primary only handles writes.

→ [Replica 1] ← read queries
[Client] → [Primary] → [Replica 2] ← read queries
↑
write queries only

■ Instagram, Twitter, and most social apps use Primary-Replica replication. Reduces primary DB load dramatically.

Key Trade-off — Replication Lag

There is a tiny delay between Primary writing data and Replica receiving it. In that window, replica has stale data.

App Type	Acceptable?	Why
Instagram likes	Yes ■	Nobody cares if like count is 1 second delayed
Twitter feed	Yes ■	Eventual consistency is fine for social feeds
Bank balance	No ■	Cannot show wrong balance even for 1 millisecond
OTP verification	No ■	Must read from Primary to guarantee freshness

ACID vs BASE

ACID (SQL Databases)

Letter	Property	Meaning	Example
A	Atomicity	Transaction is ALL or NOTHING	Bank transfer: both debit+credit happen or neither
C	Consistency	DB goes from one valid state to another, always	Account balance never goes negative
I	Isolation	Concurrent transactions don't interfere	Two users booking last seat simultaneously
D	Durability	Committed data survives crashes — written to disk	Payment confirmed = permanently saved

■ **Most important ACID property for interviews: ATOMICITY. Classic example — bank transfer. Without atomicity, money vanishes if server crashes mid-transaction.**

BASE (NoSQL Databases)

Letter	Property	Meaning
BA	Basically Available	System always responds, even if some data is stale
S	Soft state	Data may be temporarily inconsistent across nodes
E	Eventually consistent	All nodes will eventually have the same data — just not instantly

Eventually Consistent in practice:

```
You post on Instagram.
Your friend in Delhi sees it in 2 seconds.
Your friend in USA sees it in 5 seconds.
Both eventually see it – just not simultaneously.
→ Acceptable for social media.
→ NOT acceptable for banking.
```

ACID vs BASE — When to Use Which

	ACID (SQL)	BASE (NoSQL)
Consistency	Immediate — always correct	Eventual — correct after some time
Scale	Hard to scale horizontally	Built for horizontal scale
Use case	Banking, payments, e-commerce	Social feeds, analytics, catalogues
Examples	Postgres, MySQL, Oracle	MongoDB, Cassandra, DynamoDB

SQL vs NoSQL at Scale

Why SQL Struggles at Massive Scale

- Schema rigidity — ALTER TABLE on 2 billion rows can lock the whole table for hours
- Horizontal sharding is complex — maintaining ACID + JOINS across shards is very hard
- JOINS across billions of rows = full table scans = database dies
- Vertical scaling only gets you so far

When to Use MongoDB over Postgres

The correct mental model is NOT 'MongoDB = unstructured'. It is:

Choose Postgres when...	Choose MongoDB when...
Data has clear relationships (users, orders, payments)	Schema evolves frequently during development
You need JOINS across tables	Documents are self-contained (product catalogue)
ACID transactions are critical (banking)	High write throughput with horizontal scale needed
Data is highly structured and stable	Storing JSON-like documents directly

■ ■ Common misconception: 'MongoDB is for unstructured data.' More accurate: MongoDB is for FLEXIBLE or EVOLVING schemas. It absolutely has structure — it just isn't enforced rigidly.

■ Interview Answer: I'd pick MongoDB for a product catalogue where different products have different attributes (a phone has RAM, a shirt has size). Forcing that into a rigid SQL schema means either many NULL columns or complex joins. MongoDB's flexible documents are a natural fit. I'd pick Postgres for the payments table — ACID is non-negotiable there.

Interview Cheat Sheet

Answer Template — Use Every Time

"I would use _____ because _____.

The trade-off here is _____.

A real-world example of this is _____."

Must-Know Trade-offs

Topic	Key Trade-off to Mention
Caching	Speed vs staleness. Cache invalidation is hard.
Sharding	Scalability vs cross-shard query complexity and resharding pain.
Replication	Read scalability + fault tolerance vs replication lag.
Vertical scale	Simple vs single point of failure + ceiling limit.
Horizontal scale	Unlimited scale vs complexity + stateless requirement.
SQL	ACID + JOINS vs hard to scale horizontally.
NoSQL	Horizontal scale + flexible schema vs eventual consistency only.
UDP	Speed vs no delivery guarantee.
TCP	Guaranteed ordered delivery vs higher latency overhead.

Golden Rules

- Always ask: what if THIS component fails? (Load balancer, cache, DB, etc.)
- DB is always the source of truth. Cache is just a copy.
- Shard only as a last resort — exhaust index → cache → replicas → vertical first.
- Horizontal scaling needs stateless servers. Sessions go in Redis.
- For time-sensitive data, always add TTL to your eviction policy.
- For banking/payments — ACID, SQL, Primary DB reads. Never eventual consistency.
- For social/analytics — BASE, NoSQL, read from replicas is fine.

Your Progress Score Card

Week	Topic	Your Score	Target
1	Internet + APIs	76/100	85+
1	TCP vs UDP	8/10	9+
1	Scaling	7/10	9+
1	Load Balancers	6.5/10	9+
1	Caching	67/100	85+
2	Sharding + Replication + ACID	In Progress	85+

■ You are ahead of 90% of 3rd year CSE students just by being here. Keep the energy going.